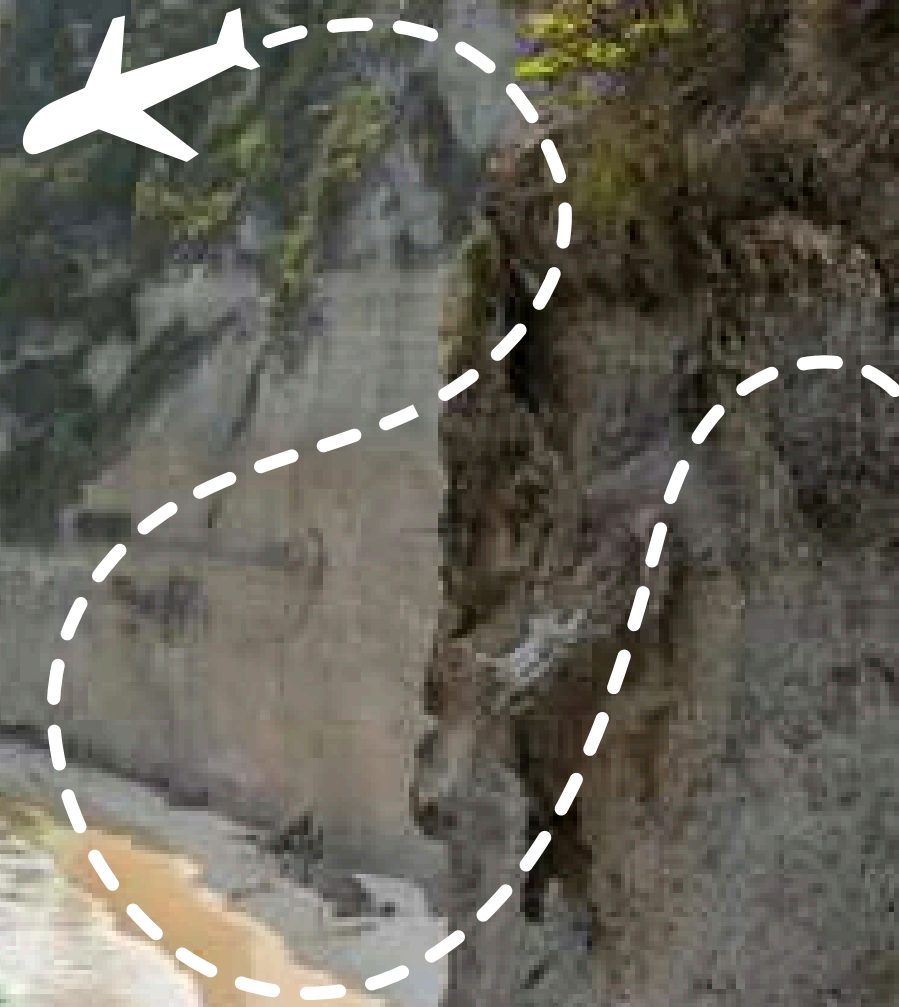


DSS NAVIGASI WISATA BALI

BY BUBADIBAKO



TIM TERBAIK KAMI

SCAN ME



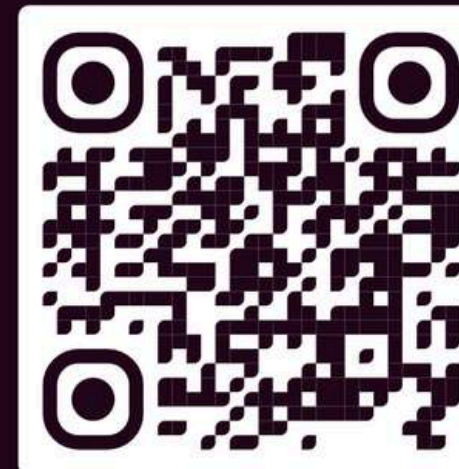
DWI MANIK
2501010107
Fotoplenger

SCAN ME



DEWA PRAMANA
2501010109
Pemadu bingung

SCAN ME



NYOMAN DIRGA
2501010116
Sopir berjalan

STUDI KASUS

Pulau Bali memiliki banyak destinasi wisata yang tersebar di berbagai wilayah seperti Kuta, Sanur, Ubud, Tanah Lot, dan GWK. Wisatawan sering mengalami kesulitan dalam menentukan rute tercepat karena banyaknya pilihan jalur dan lokasi wisata.

Untuk membantu wisatawan menentukan jalur terbaik, dibuat Sistem Pendukung Keputusan (Decision Support System / DSS) berbasis graph yang dapat menghitung rute terpendek antar lokasi wisata menggunakan algoritma Dijkstra.



TAMPILAN SISTEM DSS

PADA SISTEM INI:

- Setiap lokasi wisata direpresentasikan sebagai node/simpul.
- Jalan penghubung antar lokasi direpresentasikan sebagai edge/sisi.
- Setiap edge memiliki bobot berupa jarak tempuh dalam kilometer.
- Sistem akan menghitung total jarak terpendek dari lokasi awal menuju tujuan wisata.

SISTEM JUGA DILENGKAPI DENGAN:

- Visualisasi peta interaktif menggunakan Folium
- Marker lokasi wisata
- Tampilan jalur rute terpendek
- Estimasi waktu perjalanan
- Detail perjalanan antar lokasi



MODEL GRAPH

Representasi Graph

direpresentasikan sebagai:

$G = (V, E)$

Keterangan:

- V = Vertex / Node
- E = Edge / Jalur

Vertex (Node)

Node merupakan lokasi wisata dan wilayah di Bali.

Contoh:

- Tanah Lot
- Kuta
- GWK
- Sanur
- Denpasar
- Ubud
- Badung Regency

Edge (Sisi)

Edge adalah jalur penghubung antar lokasi.

Contoh:

- Kuta ↔ Sanur
- Sanur ↔ Ubud
- Tanah Lot ↔ GWK

MODEL GRAPH

Weight (Bobot)

Setiap edge memiliki bobot berupa jarak dalam kilometer.

Contoh:

- Kuta → Sanur = 10 KM
- Sanur → Ubud = 20 KM
- Tanah Lot → GWK = 25 KM

Jenis Graph

Project menggunakan:

- Adjacency List : Memiliki bobot jarak
- Weighted Graph : Jalur dua arah
- Undirected Graph :
Representasi graph menggunakan dictionary/list

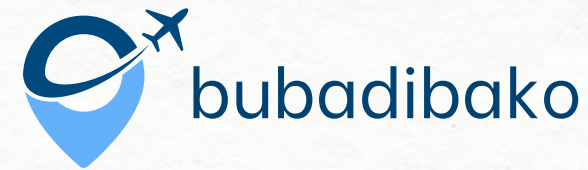
Representasi Adjacency List

```
graph = {  
  "Kuta": {  
    "Sanur": 10,  
    "Ubud": 35,  
    "Denpasar": 6  
  },  
  "Sanur": {  
    "Kuta": 10,  
    "Ubud": 20  
  }  
}
```



```
"Jembrana Regency": (-8.3650, 114.6381),
"Buleleng Regency": (-8.1120, 115.0882),
"Bangli Regency": (-8.4543, 115.3545),
"Karangasem Regency": (-8.4469, 115.6160),
"Klungkung Regency": (-8.5384, 115.4045),
"Gianyar Regency": (-8.5448, 115.3255),
"Legian": (-8.7024, 115.1738),
"Seminyak": (-8.6898, 115.1674),
"Tuban": (-8.7468, 115.1723),
"Kedonganan": (-8.7638, 115.1804),
"Abiansema": (-8.5400, 115.2070),
"Mengwi": (-8.5344, 115.1755),
"Sempidi": (-8.6207, 115.1871),
"Petang": (-8.4200, 115.2310),
"Carangsari": (-8.4120, 115.2330),
"Belok Sidan": (-8.3320, 115.2290)
```

```
road_paths = {
    tuple(sorted(("Kuta", "Sanur"))): [
        coords["Kuta"],
        (-8.717, 115.190),
        (-8.705, 115.225),
        coords["Sanur"]
    ],
    tuple(sorted(("Kuta", "Ubud"))): [
        coords["Kuta"],
        (-8.680, 115.190),
        (-8.620, 115.235),
        coords["Ubud"]
    ],
    tuple(sorted(("Sanur", "Ubud"))): [
        coords["Sanur"],
        (-8.630, 115.260),
        (-8.560, 115.260),
        coords["Ubud"]
    ]
}
```



ALGORITMA

Algoritma Dijkstra

Algoritma Dijkstra digunakan untuk mencari jalur terpendek dari lokasi awal menuju lokasi tujuan.

Cara Kerja Algoritma

1. Menentukan node awal
2. Menghitung jarak ke node tetangga
3. Memilih jalur dengan jarak terkecil
4. Mengulangi proses sampai tujuan ditemukan

Contoh Perhitungan

Mencari rute:

Kuta → Ubud

Pilihan jalur:

- Kuta → Ubud = 35 KM
- Kuta → Sanur → Ubud = 30 KM

Hasil:

Rute terbaik adalah:

Kuta → Sanur → Ubud

Total jarak:

30 KM

DEMO SISTEM

Sistem dibuat menggunakan:

- Python
- Streamlit
- Folium

Fitur Sistem

Sidebar

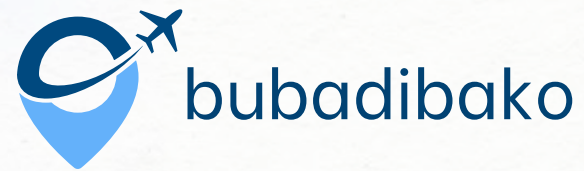
- Pilihan lokasi awal
- Pilihan tujuan
- Pengaturan zoom peta
- Tombol pencarian rute

Peta Interaktif

- Marker lokasi wisata
- Jalur antar lokasi
- Rute terpendek

Informasi Rute

- Total jarak
- Estimasi waktu
- Detail perjalanan



HASIL ANALISIS SISTEM DSS NAVIGASI WISATA BALI

1. ANALISIS SISTEM

Sistem DSS Navigasi Wisata Bali dibuat untuk membantu pengguna menentukan rute perjalanan terpendek antar lokasi wisata di Bali menggunakan konsep graph dan algoritma Dijkstra.

Pada sistem ini, setiap lokasi direpresentasikan sebagai node (simpul), sedangkan jalan penghubung antar lokasi direpresentasikan sebagai edge (sisi). Setiap edge memiliki bobot berupa jarak dalam kilometer.

Sistem mampu menghitung jalur terbaik berdasarkan total jarak paling kecil dari lokasi awal menuju lokasi tujuan.





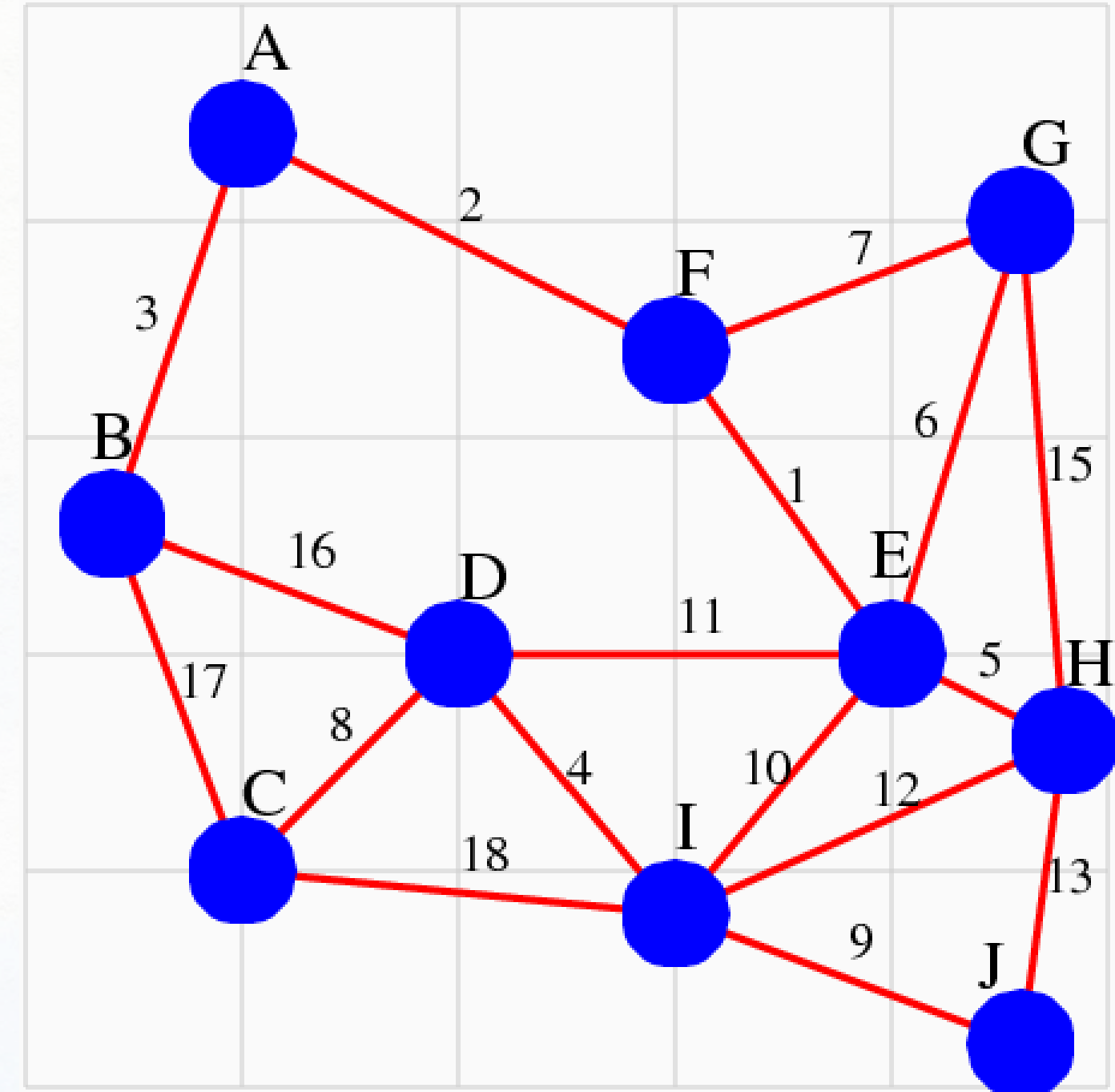
2. ANALISIS REPRESENTASI GRAPH

a. Vertex (Node)

Node pada sistem berupa lokasi wisata dan wilayah di Bali, seperti:

- Kuta
- Sanur
- Ubud
- Tanah Lot
- GWK
- Denpasar
- Badung Regency

Total node yang digunakan pada sistem adalah lebih dari 20 lokasi.





2. ANALISIS REPRESENTASI GRAPH

b. Edge (Sisi)

Edge merupakan jalur penghubung antar lokasi wisata.

Contoh:

- Kuta ↔ Sanur
- Sanur ↔ Ubud
- Tanah Lot ↔ GWK

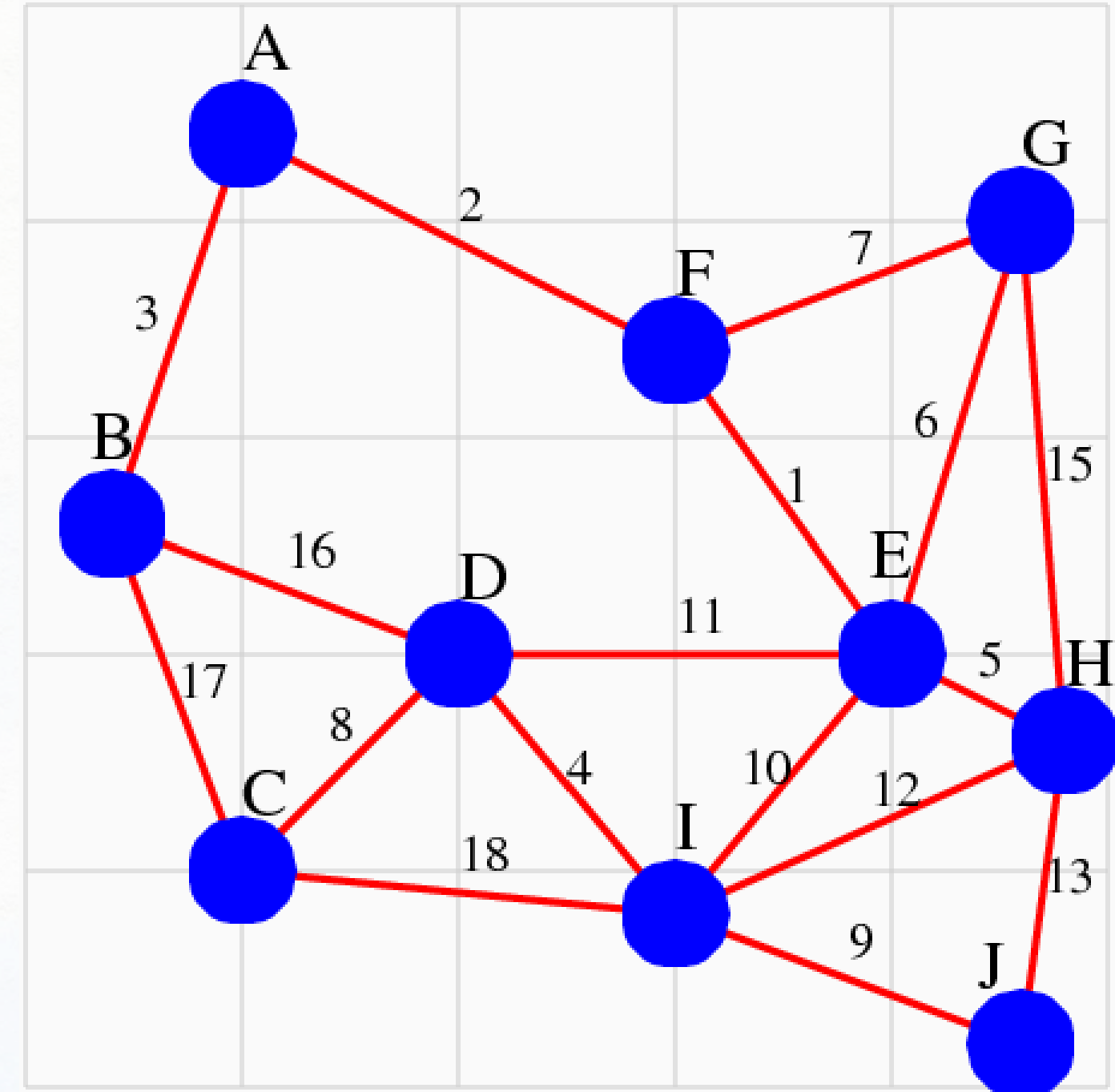
Setiap edge saling terhubung dua arah.

c. Weight (Bobot)

Bobot pada graph berupa jarak tempuh antar lokasi dalam satuan kilometer.

Contoh:

- Kuta → Sanur = 10 KM
- Sanur → Ubud = 20 KM



3. ANALISIS JENIS GRAPH

Sistem menggunakan beberapa implementasi graph, yaitu:

a. Adjacency List

Graph direpresentasikan menggunakan dictionary Python yang menyimpan daftar tetangga setiap node.

b. Weighted Graph

Setiap edge memiliki bobot berupa jarak antar lokasi.

c. Undirected Graph

Hubungan antar node bersifat dua arah.

4. ANALISIS ALGORITMA

Sistem menggunakan algoritma Dijkstra untuk menentukan jalur terpendek.

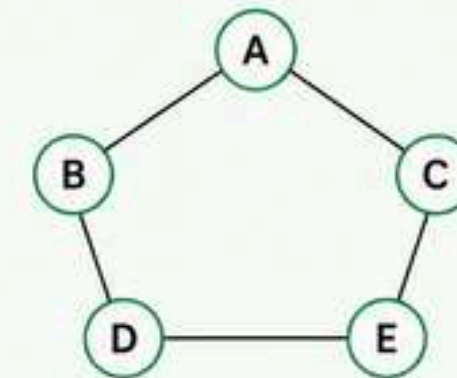
Algoritma bekerja dengan:

- Memulai pencarian dari node awal
- Menghitung total jarak ke setiap node tetangga
- Memilih node dengan jarak terkecil
- Mengulangi proses hingga mencapai tujuan

JENIS-JENIS GRAPH

1 ADJACENCY LIST

Representasi graph menggunakan struktur list, di mana setiap simpul menyimpan daftar simpul yang terhubung dengannya.

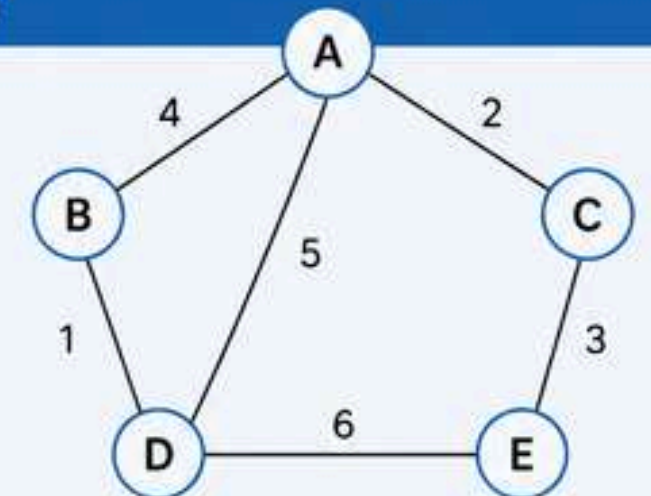


Adjacency List

A	→	B, C
B	→	A, D
C	→	A, E
D	→	B, E
E	→	C, D

2 WEIGHTED GRAPH

Graph yang setiap sisinya memiliki bobot (weight) atau nilai.

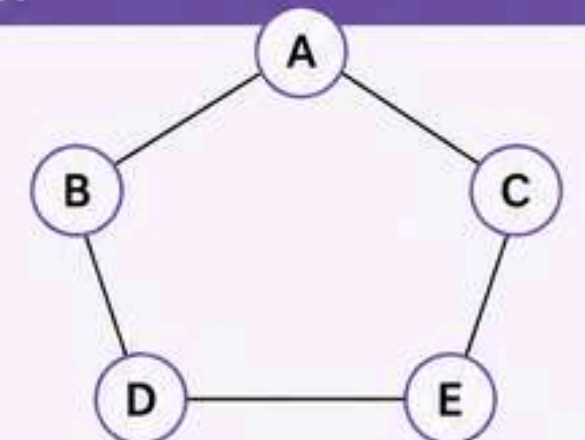


Keterangan:

Angka pada setiap sisi menunjukkan bobot dari sisi tersebut.

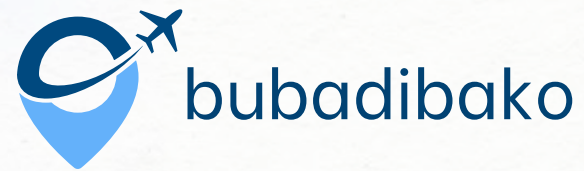
3 UNDIRECTED GRAPH

Graph yang sisinya tidak memiliki arah, artinya hubungan antara dua simpul bersifat dua arah.



Keterangan:

Tidak ada panah pada sisi, menandakan tidak ada arah.



5. ANALISIS HASIL PERHITUNGAN

Contoh pengujian:

- Lokasi awal: Kuta
- Tujuan: Ubud

Hasil sistem:

Kuta → Sanur → Ubud

Perhitungan:

- Kuta → Sanur = 10 KM
- Sanur → Ubud = 20 KM

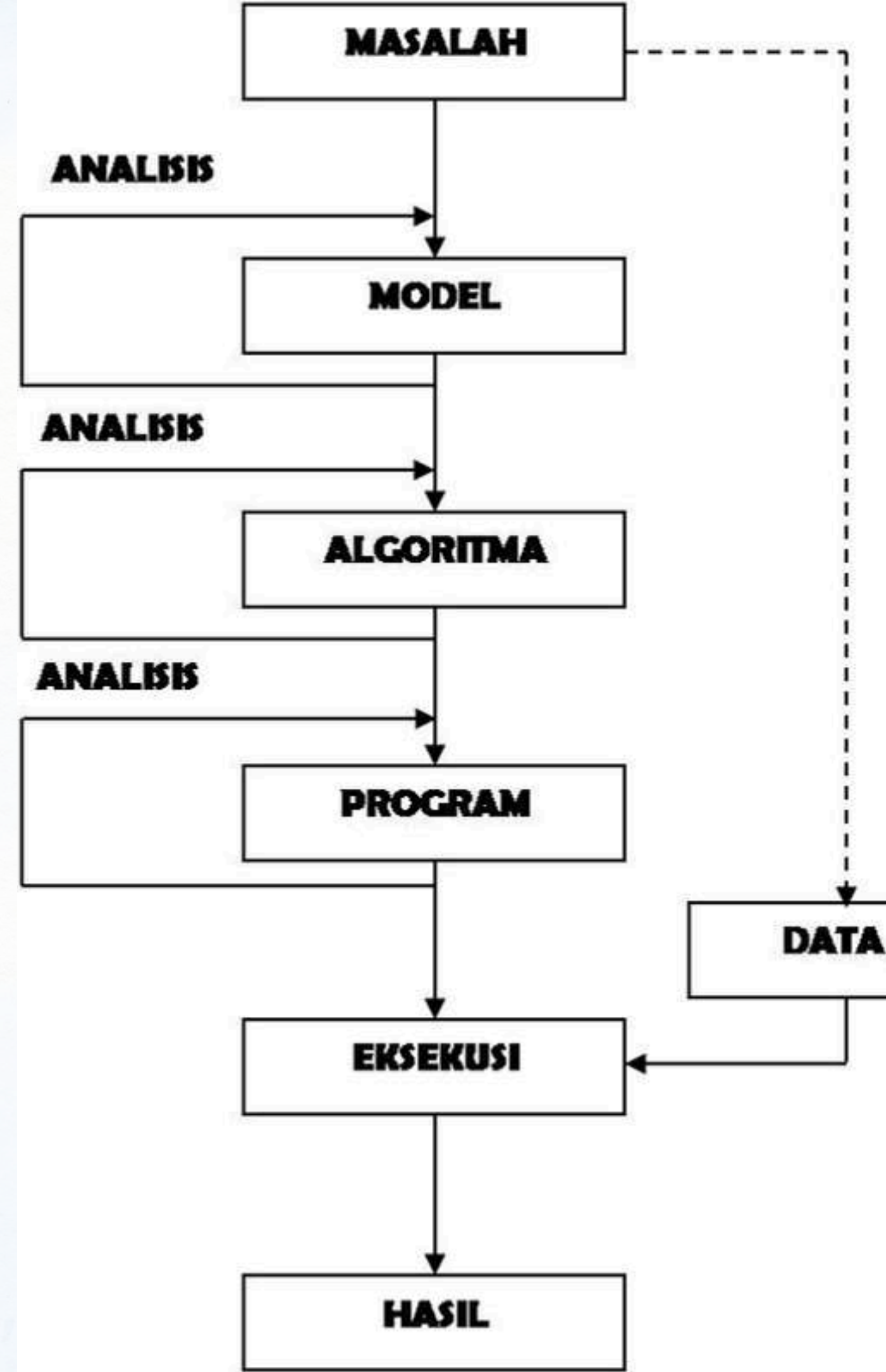
Total:

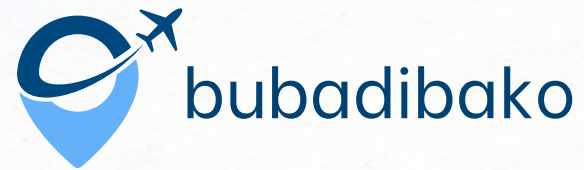
30 KM

Hasil tersebut lebih optimal dibanding jalur langsung:

Kuta → Ubud = 35 KM

Sehingga algoritma berhasil menemukan jalur paling efisien.





6. ANALISIS VISUALISASI SISTEM

Sistem menggunakan Folium untuk menampilkan:

- Marker lokasi wisata
- Jalur antar lokasi
- Rute terpendek
- Peta interaktif

Visualisasi membantu pengguna memahami rute perjalanan dengan lebih jelas.

7. HASIL ANALISIS

Sistem berhasil menerapkan:

- Adjacency List
- Weighted Graph
- Undirected Graph
- Algoritma Dijkstra



python

untuk Analisis dan
Visualisasi Data



Jalan pintas untuk
menjadi analisis data
menggunakan Python

Jubilee Enterprise

KELEBIHAN & KEKURANGAN

Kelebihan Sistem

- Mampu menentukan rute terpendek secara otomatis menggunakan algoritma Dijkstra.
- Menampilkan visualisasi peta interaktif yang mudah dipahami.
- Membantu pengguna menghemat waktu perjalanan.
- Menyediakan informasi jarak dan estimasi waktu tempuh.
- Mengimplementasikan konsep graph (Adjacency List, Weighted Graph, dan Undirected Graph) secara nyata.
- Antarmuka sederhana dan mudah digunakan.

Kekurangan Sistem

- Data lokasi dan jalur masih dimasukkan secara manual.
- Belum menggunakan data lalu lintas (traffic) secara real-time.
- Belum terintegrasi dengan Google Maps API atau GPS.
- Estimasi waktu perjalanan masih berdasarkan perhitungan sederhana dari jarak.
- Jika jumlah lokasi semakin banyak, visualisasi peta dapat menjadi lebih kompleks dan padat.
- Belum mempertimbangkan faktor lain seperti kondisi jalan, kemacetan, atau penutupan jalan.

KENDALA DAN SOLUSI

KENDALA 2

- Kendala 2
Estimasi waktu belum mempertimbangkan kemacetan.
- Solusi
Menambahkan data lalu lintas real-time.

KENDALA 1

- Kendala 1
Data lokasi dan jalur masih manual.
- Solusi
Menggunakan Google Maps API atau OpenStreetMap.

KENDALA 4

- Kendala 4
Beberapa jalur masih berupa simulasi.
- Solusi
Menggunakan koordinat GPS dan data jalan asli.

KENDALA 3

- Kendala 3
Tampilan peta menjadi padat jika node terlalu banyak.
- Solusi
Menggunakan Marker Cluster dan filter lokasi.

KESIMPULAN

Sistem DSS Navigasi Wisata Bali berhasil menerapkan konsep Graph dan Algoritma Dijkstra untuk menentukan rute terpendek antar lokasi wisata. Sistem mampu menghitung jalur dengan jarak paling minimum dan menampilkannya dalam bentuk peta interaktif yang mudah dipahami. Hasil pengujian menunjukkan bahwa sistem dapat membantu pengguna memilih rute yang lebih efisien berdasarkan jarak tempuh. Dengan demikian, sistem telah berhasil mengimplementasikan konsep graph dalam menyelesaikan permasalahan navigasi wisata di Bali.



Bali

TERIMA KASIH

BUBADIBAKO

